BOĞAZİÇİ UNIVERSITY

Department of Computer Engineering

CMPE 344: Computer Organization – Prelab 3 Report

Pipeline Waveform Analysis and Hazards on CORE-V Wally

Group ID: **FF78_G17** **Date:** November 5, 2026

Architecture: CORE-V Wally (`rv32i`, 5-Stage Pipeline) **Tools:** Verilator & GTKWave

1 Part I. Tracking the Instructions

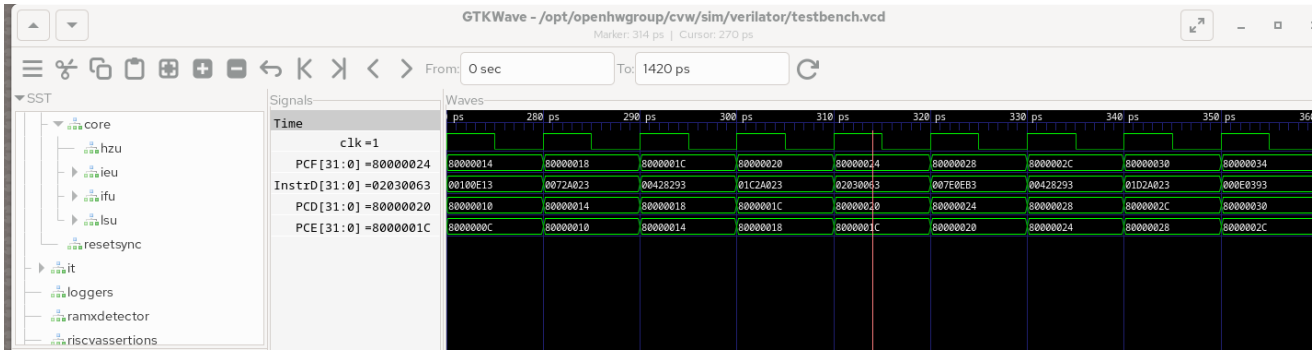


Figure 1: GTKWave waveform of the `fibonacci` simulation on CORE-V Wally (`rv32i`), showing `clk`, `PCF[31:0]`, `InstrD[31:0]`, `PCD[31:0]`, and `PCE[31:0]` with the marker placed in the clock cycle between $t = 310$ ps and $t = 320$ ps ($t = 314$ ps).

Comparing the waveform during the selected clock cycle between $t = 310$ ps and $t = 320$ ps (Figure 1) against the disassembled binary in `fibonacci.objdump` shows that the instruction currently being fetched at `PCF` and the instruction currently loaded in the Decode stage on `InstrD` are **not the same within the same clock cycle**; instead, they are separated by one clock cycle due to pipelining [1]. Specifically, during the interval $t = 310$–320 ps, the Fetch stage Program Counter is `PCF = 0x80000024`, which in `fibonacci.objdump` corresponds to the arithmetic instruction `add t4, t3, t2` (encoded as machine code `0x007e0eb3`). During that same clock cycle ($t = 310$–320 ps), however, the instruction decoded on `InstrD` is `0x02030063`, which is the machine code for the conditional branch `beqz t1, 80000040 <DONE>` located at address `0x80000020`. Looking at the preceding clock cycle ($t = 300$–310 ps) on the waveform confirms that `PCF` was `0x80000020` one cycle earlier; on the rising clock edge at $t = 310$ ps, that address and its fetched instruction were latched into the Decode stage registers, producing `PCD = 0x80000020` and `InstrD = 0x02030063` during $t = 310$–320 ps (while the earlier store instruction at `0x8000001c` moved into the Execute stage at `PCE = 0x8000001c`). Likewise, the instruction fetched from `PCF = 0x80000024` during $t = 310$–320 ps is latched across the IF/ID pipeline register on the rising clock edge at $t = 320$ ps and appears on `InstrD` as `0x007e0eb3` alongside `PCD = 0x80000024` during the next clock cycle ($t = 320$–330 ps), while `beqz t1, DONE` advances to the Execute stage (`PCE = 0x80000020`). Because inter-stage pipeline registers (`PCD` and `PCE`) pass instruction addresses and machine words forward by one stage on each rising clock edge, `PCF` always reflects the instruction being fetched from memory in the current cycle, whereas `InstrD` always holds the instruction at `PCD` that was fetched one cycle earlier.

2 Part II. Unexpected Instructions

In `fibonacci.objdump`, the loop body ends at address `0x8000003c` with the unconditional jump instruction `j 80000020 <LOOP>` (machine code `0xfe5ff06f`), and the next sequential address in program memory is `0x80000040` under the label `DONE`, which holds the self-looping halt instruction `j 80000040 <DONE>` (machine code `0x0000006f`). In Wally's 5-stage pipeline, branch outcomes and jump target addresses are not determined until the instruction reaches the Execute (`E`) stage, two clock cycles after it is fetched [1]. Because the Instruction Fetch Unit (`ifu`) must fetch an instruction on every clock cycle to keep all five pipeline stages utilized and sustain a throughput close to one instruction per cycle, it cannot stall and wait two cycles after every instruction just to check whether control flow will change. Instead, the `ifu` uses speculative sequential fetching: on each clock cycle, it assumes no branch or jump is taken (*predict-not-taken*) and advances the Program Counter sequentially to `PCNextF = PCF + 4` [2]. Consequently, after `j LOOP` is fetched during $t = 370$–380 ps (`PCF = 0x8000003c`) and latched into the Decode stage during $t = 380$–390 ps (`PCD = 0x8000003c`, `InstrD = 0xfe5ff06f`), the Fetch stage automatically increments `PCF` to the next static memory address `0x80000040` and fetches `0x0000006f` (`DONE`). In the following cycle ($t = 390$–400 ps), as `j LOOP` enters the Execute stage (`PCE = 0x8000003c`), the sequentially fetched `0x0000006f` instruction enters the Decode stage (`PCD = 0x80000040`, `InstrD = 0x0000006f`) while `PCF` increments again to `0x80000044`. This speculative fetching mechanism is provided because the vast majority of executed instructions are sequential and conditional branches such as `beqz t1, DONE` (at `0x80000020`) are not taken during the first 10 iterations of the loop; fetching `PCF + 4` every cycle allows sequential code and untaken branches to execute without any stall penalty.

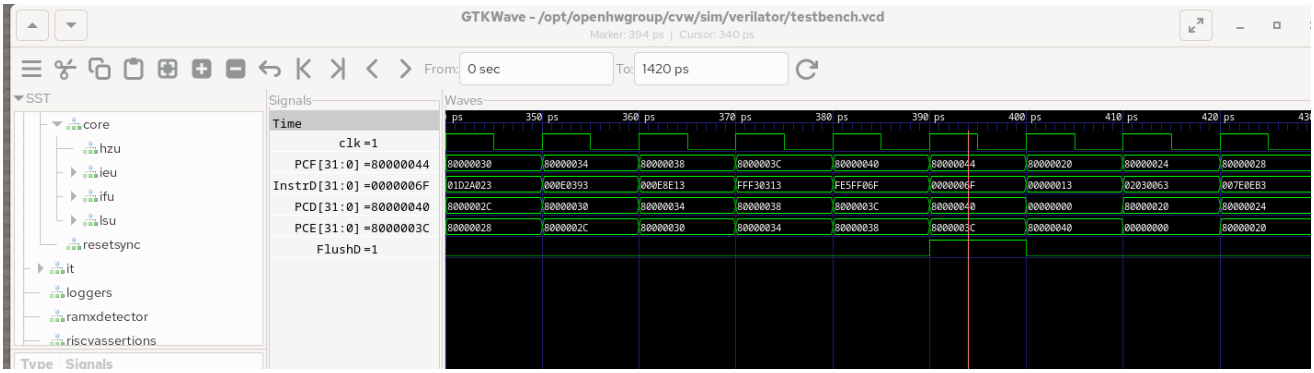


Figure 2: GTKWave waveform showing the wrong-path instruction `InstrD = 0000006F` (`DONE`) and `FlushD = 1` during $t = 390$–400 ps ($t = 394$ ps marker), followed by the pipeline flush and jump redirection in $t = 400$–410 ps.

During the clock cycle $t = 390$–400 ps, the unconditional jump `j 80000020 <LOOP>` is in the Execute stage (`PCE = 0x8000003c`), where the processor resolves the jump condition and computes the target address `0x80000020`. Because the two instructions that were speculatively fetched after the jump (currently at `PCF = 0x80000044` in Fetch and `PCD = 0x80000040`, `InstrD = 0x0000006f` in Decode) lie on the wrong execution path and must not be allowed to alter architectural state, the Hazard Unit (`hzu`) asserts `FlushD = 1` (as well as `FlushE = 1`) throughout $t = 390$–400 ps [1]. At the clock cycle right after `FlushD` is high ($t = 400$–410 ps, Figure 2), three actions occur on the rising clock edge at $t = 400$ ps: first, the asserted `FlushD` signal synchronously clears the IF/ID pipeline registers, discarding the wrong-path instruction fetched from `0x80000044`, resetting `PCD` to `0x00000000`, and replacing `InstrD` with a `nop` bubble (`addi x0, x0, 0`, encoded as `0x00000013`); second, the wrong-path `0x0000006f` instruction advancing from Decode into Execute (`PCE = 0x80000040`) is simultaneously flushed into a `nop` (whose address becomes `0x00000000` in the next cycle) so that it performs no register writes or memory operations; and third, `PCF` is redirected to the jump target address `0x80000020` (`LOOP`) to fetch the loop header instruction `beqz t1, DONE` (`0x02030063`), while `FlushD` deasserts back to `0`. In the subsequent cycle ($t = 410$–420 ps), `0x02030063` is latched into `InstrD` with `PCD = 0x80000020`, resuming execution at the start of the second loop iteration.

3 Part III. Data Hazard

In the Part III template program, `lw x6, 0(x5)` at `0x80000014` loads the first array element (`arr[0] = 1111`) into register `x6` and is separated by a single `nop` (`0x80000018`) from the dependent instruction `add x8, x6, x6` at `0x8000001c`. This sequence creates a **Read-After-Write (RAW) data hazard** on `x6`. As shown in Figure 3, during $t = 290$–300 ps, `lw x6, 0(x5)` is in Execute (`PCE = 0x80000014`, `Rs1E = 05`, `RdE = 06`) while `add x8, x6, x6` is fetched (`PCF = 0x8000001c`). During $t = 300$–310 ps, when `add x8, x6, x6` is in Decode (`PCD = 0x8000001c`) reading `x6` from the register file, `lw x6, 0(x5)` is still in the Memory stage (`RdM = 06`) reading data memory, so the register file still outputs the old value of `x6` (0). In the third clock cycle shown ($t = 310$–320 ps), `lw x6, 0(x5)` reaches Writeback (`RdW = 06`) at the exact same clock cycle that `add x8, x6, x6` enters Execute (`PCE = 0x8000001c`, `Rs1E = 06`, `Rs2E = 06`, `RdE = 08`).

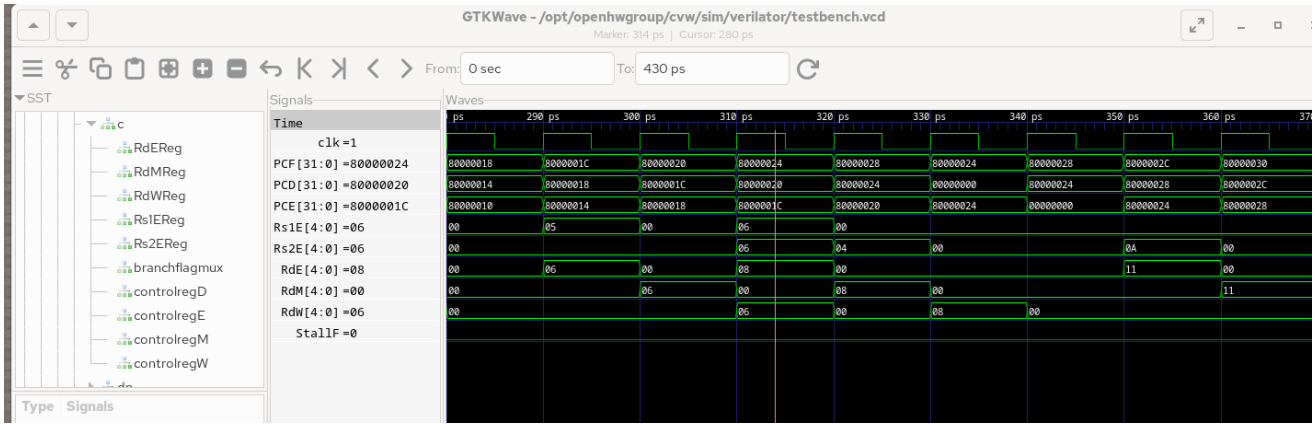


Figure 3: GTKWave waveform for Part III at $t = 310$–320 ps ($t = 314$ ps marker), where `add x8, x6, x6` is in the Execute stage (`PCE = 8000001C`, `Rs1E = 06`, `Rs2E = 06`, `RdE = 08`) in the same cycle that `lw x6, 0(x5)` is in the Writeback stage (`RdW = 06`) with `StallF = 0`.

The fundamental mechanism that allows an instruction in the Execute stage to use a register value in the same clock cycle that it is changing in the Writeback stage, without stalling the pipeline (`StallF = 0`), is **Data Forwarding (also called Bypassing)** [1, 2]. Without forwarding, the ALU in Execute would only receive operand values read from the register file during Decode, which are stale whenever an earlier instruction in Memory or Writeback has not yet written its result back to the register file. Data forwarding resolves this hazard in hardware by routing direct combinational bypass paths from the outputs of the Memory (`M`) and Writeback (`W`) stages back to 3-to-1 multiplexers placed at the ALU inputs in the Execute (`E`) stage [1]. The Hazard Unit compares the Execute source registers (`Rs1E` and `Rs2E`) against the destination registers in Memory (`RdM`) and Writeback (`RdW`):

$$\texttt{ForwardAE} = \begin{cases} \texttt{10} & \text{if } (\texttt{Rs1E} == \texttt{RdM}) \wedge (\texttt{Rs1E} \neq 0) \wedge \texttt{RegWriteM} \text{ (Forward ALU result from Memory stage)} \\ \texttt{01} & \text{else if } (\texttt{Rs1E} == \texttt{RdW}) \wedge (\texttt{Rs1E} \neq 0) \wedge \texttt{RegWriteW} \text{ (Forward result from Writeback stage)} \\ \texttt{00} & \text{otherwise (Use register file value read in Decode stage)} \end{cases}$$

(with identical logic generating `ForwardBE` for `Rs2E`). During $t = 310$–320 ps in Figure 3, the instruction in Memory is `nop` (`RdM = 00`), whereas the instruction in Writeback is `lw x6, 0(x5)` (`RdW = 06`, `RegWriteW = 1`). Because `Rs1E = 06 == RdW` and `Rs2E = 06 == RdW`, the Hazard Unit sets `ForwardAE = 01` and `ForwardBE = 01`. Both ALU input multiplexers bypass the old register file values from Decode and select the Writeback result bus (`ResultW = 1111`) directly, computing $1111 + 1111 = 2222$ during $t = 310$–320 ps with no stall (`StallF = 0`). (When a consumer instruction is in *Decode* while the producer is in *Writeback*, a 5-stage pipeline also uses **split-phase register file access**, writing on the falling clock edge and reading combinationally in the second half of the cycle [1, 2]; when the consumer is in *Execute* as in Figure 3, **Writeback-to-Execute forwarding** supplies the operand directly to the ALU.)

4 Part IV. Load-Use Hazard

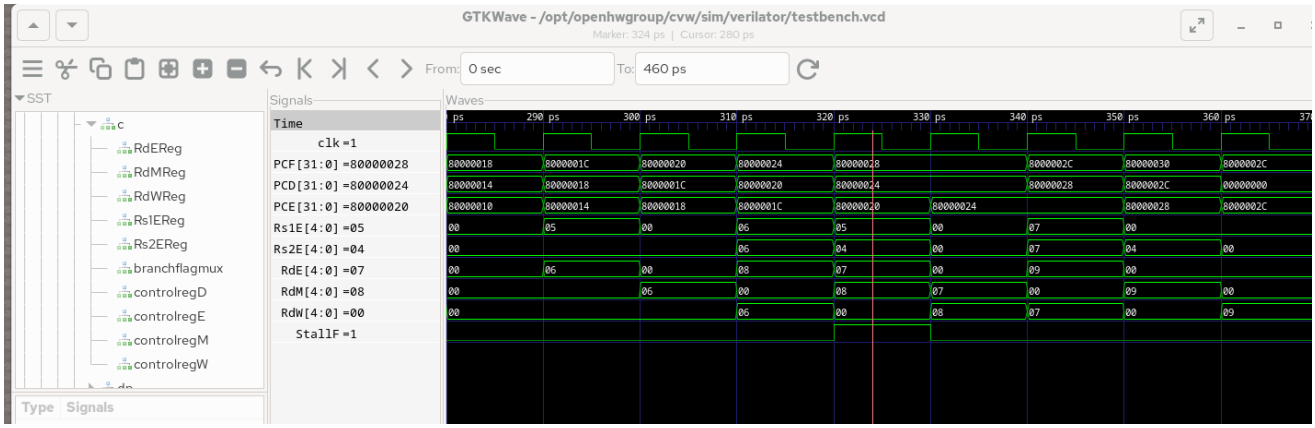


Figure 4: GTKWave waveform of `prelab3_FF78_G17.s` across $t = 320$–350 ps ($t = 324$ ps marker): `StallF = 1` is asserted during $t = 320$–330 ps when `lw x7, 4(x5)` is in Execute (`PCE = 80000020`, `RdE = 07`) and `add x9, x7, x7` is in Decode (`PCD = 80000024`), stalling the pipeline during $t = 330$–340 ps.

To implement the most simplistic load-use hazard inside the template block of `prelab3_FF78_G17.s`, we added `lw x7, 4(x5)` (at `0x80000020`, loading the second array element `arr[1] = 2222` into `x7`) immediately followed by `add x9, x7, x7` (at `0x80000024`, using `x7` as both `Rs1` and `Rs2`) with no `nop` between them. Unlike an arithmetic instruction whose result is ready at the end of the Execute (`E`) stage, a load instruction (`lw`) only computes the effective memory address (`x5 + 4`) in Execute and does not receive the loaded word from data memory until the end of the **Memory (`M`)** stage [1, 2]. Because the loaded data is not yet available at the end of Execute, forwarding alone cannot supply `x7` to an instruction that enters Execute in the very next cycle, requiring a one-cycle stall. In Figure 4, during $t = 320$–330 ps ($t = 324$ ps marker), `lw x7, 4(x5)` is in Execute (`PCE = 0x80000020`, `Rs1E = 05`, `RdE = 07`), the dependent `add x9, x7, x7` is in Decode (`PCD = 0x80000024`) requesting `x7`, and `j END` is in Fetch (`PCF = 0x80000028`). Detecting that the instruction in Execute is a load whose destination register (`RdE = 07`) matches the Decode source operands, the Hazard Unit asserts `StallF = 1` (along with `StallD = 1` and `FlushE = 1`) during $t = 320$–330 ps [1]. Consequently, in the next clock cycle ($t = 330$–340 ps), `PCF` stays held at `0x80000028` (preventing a new instruction fetch) and `PCD` stays held at `0x80000024` (keeping `add x9, x7, x7` stalled in Decode for one cycle), while a `nop` bubble (`Rs1E = 00`, `Rs2E = 00`, `RdE = 00`) is inserted into Execute as `lw x7, 4(x5)` moves to the Memory stage (`RdM = 07`). At $t = 340$–350 ps, `lw x7, 4(x5)` enters Writeback (`RdW = 07`), `StallF` returns to `0`, and `add x9, x7, x7` enters Execute (`PCE = 0x80000024`, `Rs1E = 07`, `Rs2E = 07`, `RdE = 09`), receiving the loaded value of `x7` from Writeback via forwarding.

References

[1] S. L. Harris and D. Harris, *Digital Design and Computer Architecture: RISC-V Edition*, Morgan Kaufmann / Elsevier, 2021, Section 7.5 (“Pipelined Processor” and “Hazards”), pp. 426–462.

[2] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*, 2nd ed., Morgan Kaufmann, 2020, Sections 4.7–4.8 (“Data Hazards: Forwarding versus Stalling” and “Control Hazards”), pp. 303–325.